

Project Report Checkpoint 2

Data Query

Spatial Gene Expression from Histology

Mobai (James) Jiang, Junewoo (June) Koo, Bobby Wang, Jiayin (Moria) Li

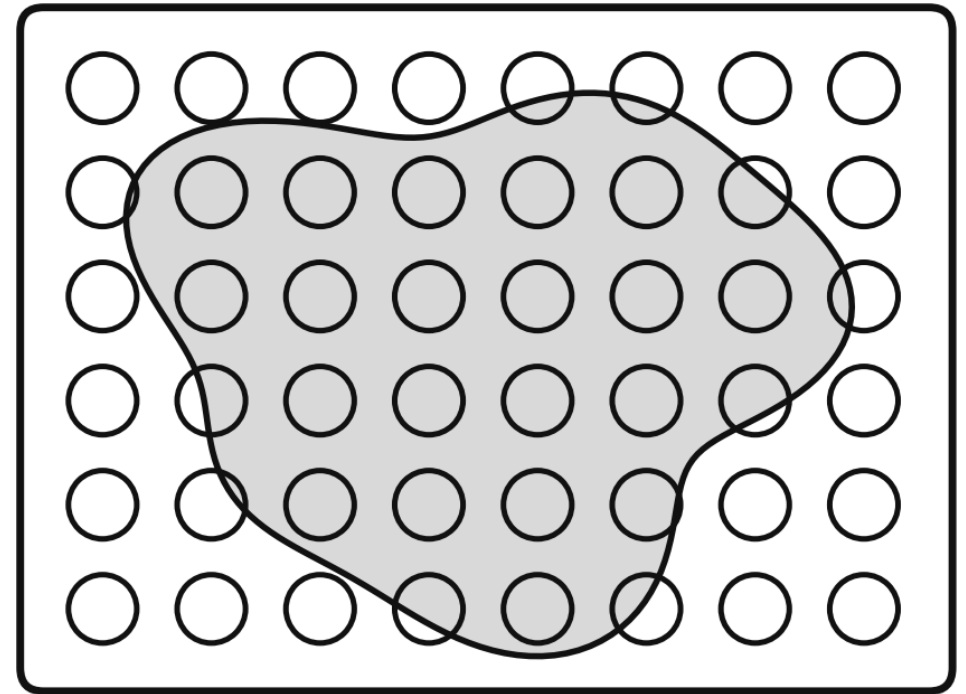
GitHub: <https://github.com/Accord985/Hist2ST>

Our project in one sentence

- Can we predict the genes that are active at each spot from what tissue looks like under a microscope?
- Input: A small H&E image patch at one spot
- Output: How much each gene is expressed at that spot

What is a spot?

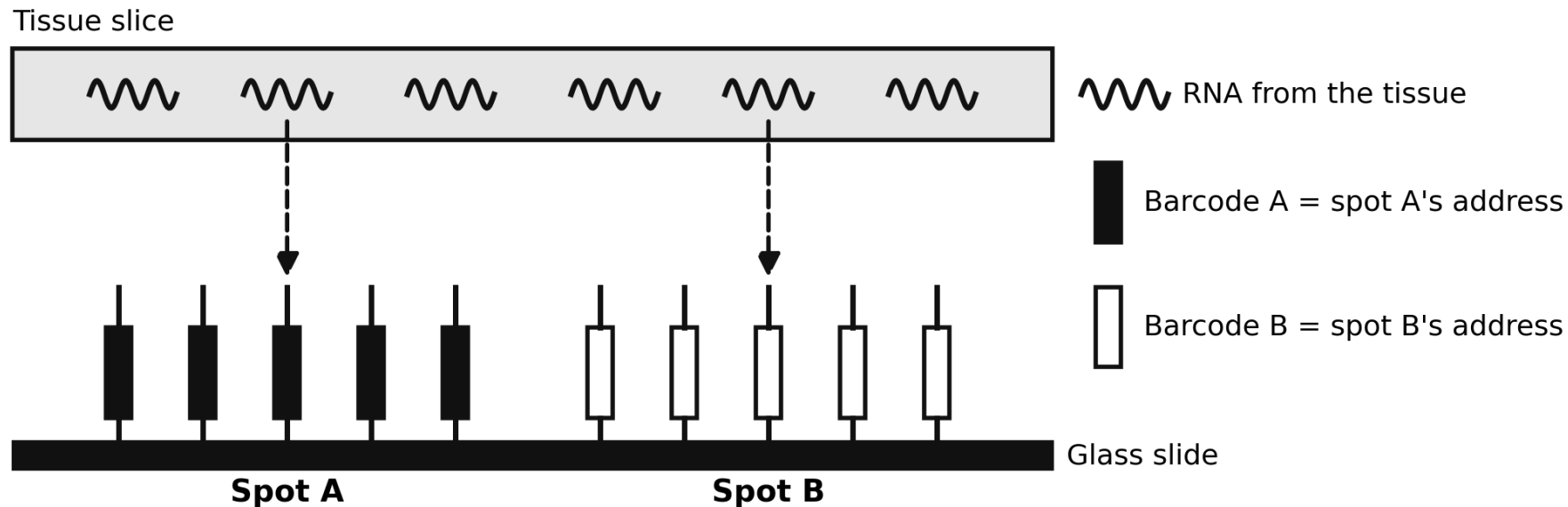
- Spatial transcriptomics (10x Visium)
 - A thin slice of tissue is placed on a special glass slide
 - The slide has a grid of thousands of tiny spots ($\sim 55\text{ }\mu\text{m}$, about 1–10 cells)
 - Each spot catches the RNA from the cells right above it and tags it with the spot's barcode
 - The same slice is stained with H&E and photographed



Gray = tissue slice · each circle = one spot

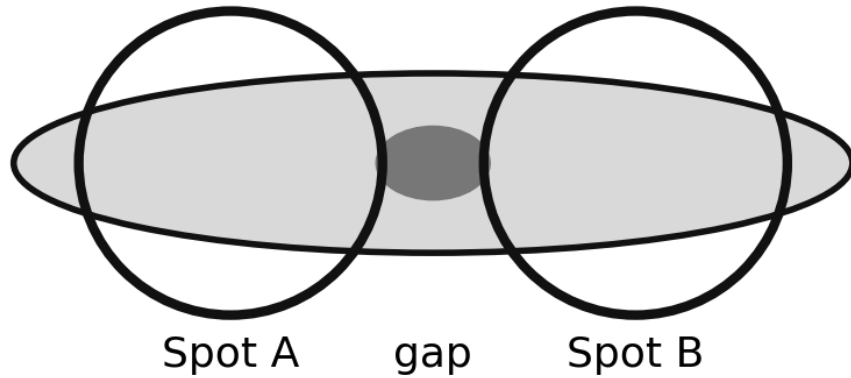
Barcode = the spot's address

- Made by the lab, not the cell: DNA “catchers” are pre-printed on the slide
- All catchers in one spot share one barcode (~16 letters of A, T, G, C)
- RNA falls onto a catcher → its DNA copy carries that barcode
- After sequencing, the barcode tells us which spot the RNA came from

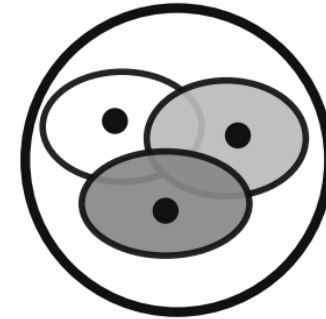


One spot = a mix of cells

Cell bigger than a spot



Cells overlapping



One spot, 3 cells → one pooled reading

- The “true answer” we predict is a blurry average of several cells, not one clean cell

Every spot gives us two things

- A picture: the H&E patch around that spot (224 × 224 pixels)
- A list of numbers: how much of each gene's RNA was caught there
- Like a satellite photo of a block + the weather station on that block
 - Can we guess the station's reading from the photo alone?

Our data: HEST

- Made by the Mahmood Lab (Harvard Medical School / Mass General Brigham)
- HEST-1k (dataset): 1,276 samples of paired H&E images and spot-level gene expression, in one standard format
- HEST-Benchmark (test): which pretrained model gives the best embeddings?
 - Embedding → PCA → ridge regression
 - Scores (Pearson r): about 0.2–0.67 by cancer type, ~0.42 on average

Getting the data

- Hosted on Hugging Face: huggingface.co/datasets/MahmoodLab/hest (free, but gated)
 - 1. Request access: log in and accept the dataset's terms
 - 2. Make a read-only token and log in with it
 - 3. Download only one sample's small files, not the multi-GB whole-slide images
- Full notebook: github.com/Accord985/Hist2ST

```
snapshot_download(repo_id="MahmoodLab/hest", repo_type="dataset",  
                  allow_patterns=["patches/*TENX95*", "st/*TENX95*", ...])
```

What's in the download

Folder / file	What's inside	Used for
wsis/	Full whole-slide H&E image (multi-GB)	Rarely needed
patches/ (.h5)	224×224 patch for each spot + its barcode and position	Model input
st/ (.h5ad)	Gene table: spots × genes, with each spot's barcode	Target (true answer)
thumbnails/, spatial_plots/	Small preview pictures	Just for looking
HEST_v1_1_0.csv	One row per sample: organ, species, technology, cancer type	Choosing samples

The barcode links them: same spot ID in patches/ and st/ → picture matched to its gene values

One sample: TENX95

- Human breast tissue, measured with Xenium
- 1 tissue slide → 11,845 spots (one patch each) → 541 gene values per spot
- Xenium measures single RNA molecules, so HEST draws imaginary Visium-sized pseudo-spots
 - Every sample then has the same format: one patch + one row of gene values per spot

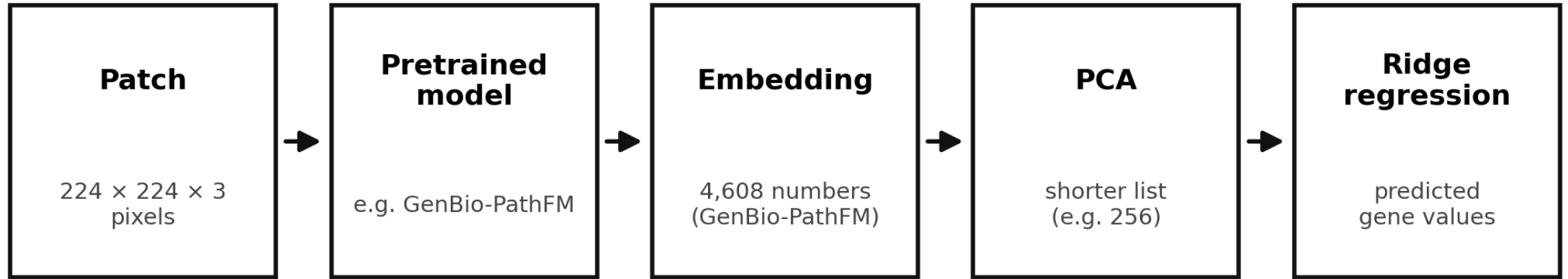
What the gene data looks like

Barcode (spot ID)	Gene 1	Gene 2	Gene 3	... Gene 541
spot_00001	12	0	3	...
spot_00002	8	1	0	...
...	...	...	...	...
spot_11845	15	0	5	...

Example values for illustration

- One row = one spot · one column = one gene
- The barcode links each row to its image patch
- We predict a smaller chosen set (e.g. 50 of the 541 genes)

From pixels to gene values



- Training input: the embedding of each patch (after PCA), a list of numbers that summarizes only the image
- Training target: the real measured gene values at that spot

Sample catalog (HEST_v1_1_0.csv)

...	id	organ	species	st_technology	oncotree_code
0	TENX159	Brain	Mus musculus	Xenium	Missing value
1	TENX158	Skin	Homo sapiens	Xenium	SKCM
2	TENX157	Prostate	Homo sapiens	Xenium	PRAD
3	TENX156	Bowel	Homo sapiens	Visium HD	COAD
4	TENX155	Bowel	Homo sapiens	Visium HD	COAD
5	TENX154	Bowel	Homo sapiens	Visium HD	COAD
6	TENX153	Bowel	Homo sapiens	Visium HD	Missing value
7	TENX152	Bowel	Homo sapiens	Visium	COAD
8	TENX149	Bowel	Homo sapiens	Xenium	COAD
9	TENX148	Bowel	Homo sapiens	Xenium	COAD

20 rows x 5 cols 10 per page

Page 1 of 2

Search, Grid, Download, More icons

- One row = one tissue slide (1,276 in total)
- Columns: sample id, organ, species, technology, cancer type (oncotree code)
- Used to choose which samples to download

Breast samples only

	id	organ	species	st_technology
153	TENX99	Breast	Homo sapiens	Xenium
154	TENX98	Breast	Homo sapiens	Xenium
155	TENX97	Breast	Homo sapiens	Xenium
156	TENX96	Breast	Homo sapiens	Xenium
157	TENX95	Breast	Homo sapiens	Xenium
158	TENX94	Breast	Homo sapiens	Xenium
183	TENX68	Breast	Homo sapiens	Visium
193	TENX53	Breast	Homo sapiens	Visium
203	TENX39	Breast	Homo sapiens	Visium
209	TENX24	Breast	Homo sapiens	Visium

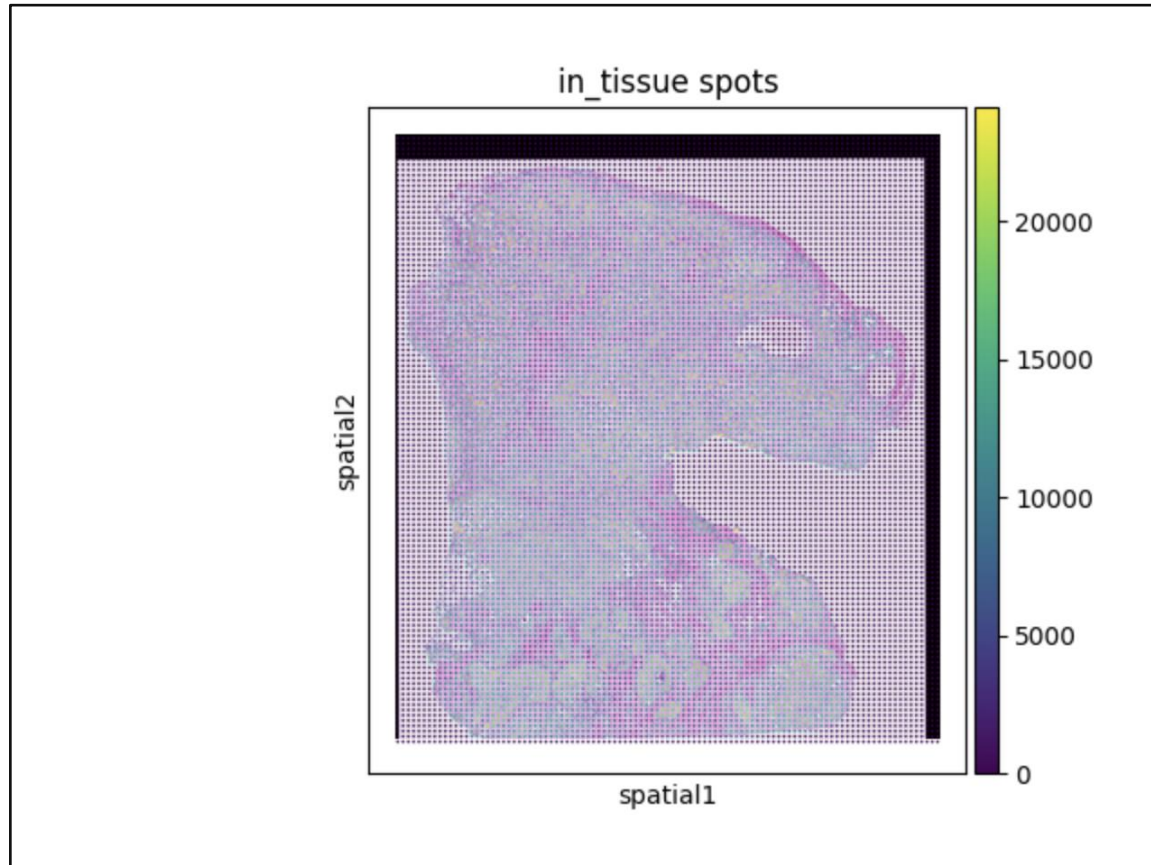
125 rows x 4 cols 10 per page

<< < Page 1 of 13 > >>

🔍 📊 📄 ...

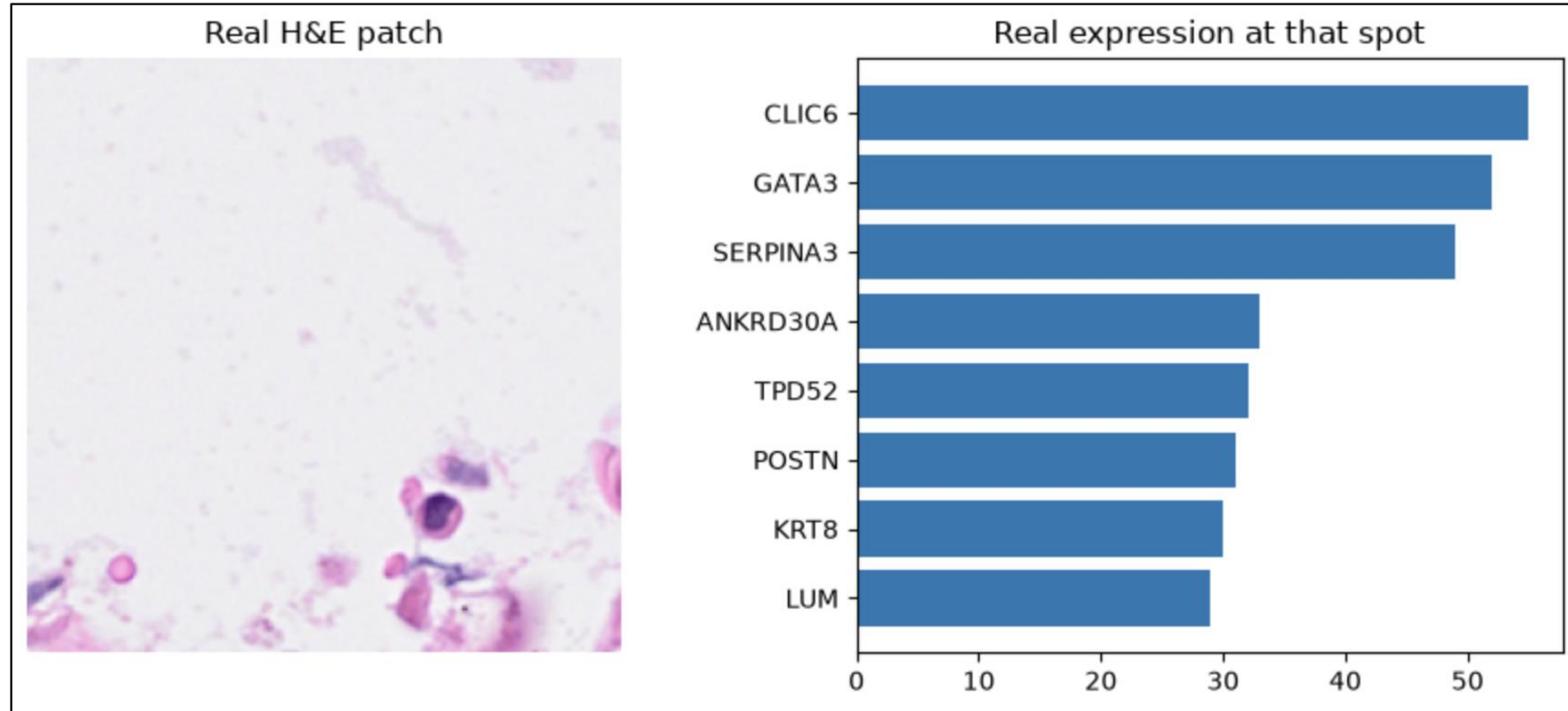
- 125 breast samples, TENX95 among them
- Mixed technologies (Xenium, Visium, ...) → mix them or pick one?

TENX95: the whole slide



- Background: the H&E image
- Dots: ~11,845 pseudo-spots
- spatial1 / spatial2: x and y position
- Color bar: total RNA counts per spot (0 to ~24,000)
- Black strip: spots with no tissue (~0 counts)

One spot up close



Left: the image patch (model input) · Right: its top 8 genes (the answer to predict)

Slide pre-processing

- We are able to implement GenBio – PathFM
- The result is 1 a by 4608 list
- Next step is a pipeline that mass produce embeddings

[20]

✓ 0s



```
from PIL import Image

image_path = "/content/TRIDENT/tests/HIST_files/TENX94_patch_00035.png"

image = Image.open(image_path).convert("RGB")

x = transform(image).unsqueeze(0).to(device)

print("Input shape:", x.shape)

with torch.inference_mode():
    embedding = model(x)

print("Embedding shape:", embedding.shape)
```



```
... Input shape: torch.Size([1, 3, 224, 224])
    Embedding shape: torch.Size([1, 4608])
```

[21]

✓ 0s



```
print(embedding)
```



```
... tensor([[ -0.7285, -0.1346,  0.2831, ..., -0.4297, -0.3189, -0.9732]],
          device='cuda:0')
```

